

Arquiteturas de Software

Objetivo do arquivo

Resumo objetivo dos principais estilos arquiteturais vistos na disciplina.

O foco é lembrar estrutura, vantagens, desvantagens e quando cada arquitetura faz sentido em prova.

O que é arquitetura de software

- Definição: organização geral do sistema, mostrando como seus módulos, componentes, serviços e camadas se relacionam.
- Por que é importante: define a base estrutural do software e influencia manutenção, escalabilidade, testes e evolução.
- Relação com projeto orientado a objetos: arquitetura e POO trabalham juntas; a arquitetura organiza o sistema como um todo e a POO organiza partes menores.
- Relação com manutenção, reuso, escalabilidade e acoplamento: uma boa arquitetura reduz acoplamento, melhora coesão, facilita reuso, organiza responsabilidades e ajuda o sistema a crescer com menos impacto.

Conceitos importantes

Camadas

- Definição: divisão hierárquica do sistema em níveis, em que uma camada usa serviços da camada abaixo.
- Exemplo: interface, regra de negócio e acesso a dados em camadas separadas.
- Vantagem: reduz complexidade, facilita entendimento e favorece reuso.
- Desvantagem: pode adicionar indireção e excesso de estrutura.

Componentes

- Definição: unidades do sistema com responsabilidade bem definida.
- Exemplo: módulo de pagamento, módulo de relatórios, módulo de autenticação.
- Vantagem: separa responsabilidades e facilita manutenção.
- Desvantagem: coordenação entre componentes pode ficar mais difícil.

Cliente e servidor

- Definição: um lado consome serviços e o outro oferece serviços.
- Exemplo: navegador acessando um servidor web.
- Vantagem: centraliza serviços e simplifica distribuição de recursos.
- Desvantagem: depende da comunicação pela rede e pode criar gargalos.

Serviços

- Definição: partes do sistema que expõem funcionalidades para serem usadas por outras partes.
- Exemplo: serviço de autenticação, serviço de pedidos, serviço de notificações.

- Vantagem: separa responsabilidades e facilita reuso.
- Desvantagem: aumenta o número de integrações e a complexidade de comunicação.

Monolito

- Definição: sistema em que módulos rodam no mesmo processo ou aplicação.
- Exemplo: uma aplicação desktop ou backend único com vários módulos internos.
- Vantagem: implantação e execução mais simples.
- Desvantagem: pode ficar difícil de evoluir, escalar e manter em sistemas grandes.

Distribuição

- Definição: sistema cujas partes rodam em processos, máquinas ou serviços diferentes.
- Exemplo: microsserviços, cliente-servidor, publish/subscribe.
- Vantagem: melhora escalabilidade e autonomia de partes do sistema.
- Desvantagem: aumenta complexidade, latência e dificuldade de observabilidade.

Anti-padrão arquitetural

- Definição: estilo ruim de organização que revela problemas sérios de estrutura.
- Exemplo: Big Ball of Mud.
- Vantagem: não é vantagem; é sinal de problema arquitetural.
- Desvantagem: dependências circulares, baixa modularidade e muita bagunça.

Arquiteturas vistas na disciplina

Arquitetura monolítica

- Definição: sistema em que tudo roda como uma única aplicação/processo em tempo de execução.
- Como funciona: os módulos existem internamente, mas compartilham o mesmo processo principal.
- Componentes principais: módulos internos, frontend e backend integrados, banco de dados único ou central.
- Quando usar: sistemas menores, times pequenos ou quando a prioridade é simplicidade inicial.
- Vantagens:
 - implantação mais simples;
 - menor complexidade inicial;
 - comunicação interna mais direta;
 - mais fácil começar o projeto.
- Desvantagens:
 - escalabilidade horizontal mais limitada;
 - release mais lenta;
 - alto impacto de mudanças;
 - risco de virar "bagunça" com o tempo;
 - single point of failure.
- Exemplo prático: uma aplicação desktop com interface, regras de negócio e acesso a dados no mesmo programa.
- Relação com princípios de projeto: pode ter boa coesão interna, mas tende a aumentar acoplamento se o crescimento não for bem controlado.

- Frase curta para prova: "Monolito é quando tudo roda como uma única aplicação, o que simplifica o início, mas pode dificultar evolução e escala."

Arquitetura em camadas

- Definição: sistema organizado em níveis hierárquicos, em que uma camada usa serviços da camada imediatamente abaixo.
- Como funciona: cada camada concentra um tipo de responsabilidade e conversa com a camada vizinha.
- Componentes principais: camada de apresentação, camada de lógica/regra de negócio e camada de dados.
- Quando usar: quando se quer organizar o sistema por responsabilidades e reduzir dependências diretas.
- Vantagens:
 - divide para conquistar;
 - reduz complexidade;
 - facilita entendimento;
 - facilita troca de tecnologias em uma camada;
 - favorece reuso de camadas.
- Desvantagens:
 - pode adicionar indireção;
 - pode ser rígida se houver dependência demais entre camadas;
 - pode ficar pesada para sistemas simples.
- Exemplo prático: um sistema web com interface, serviços de negócio e persistência separados.
- Relação com princípios de projeto: reduz acoplamento, melhora coesão e separa responsabilidades.
- Frase curta para prova: "Arquitetura em camadas organiza o sistema hierarquicamente e reduz a complexidade separando responsabilidades."

Arquitetura de duas camadas

- Definição: variação em que há uma camada cliente e uma camada de servidor de banco de dados.
- Como funciona: o cliente concentra interface e parte da lógica; o servidor mantém os dados.
- Componentes principais: cliente, servidor de banco de dados.
- Quando usar: sistemas mais simples, especialmente quando a aplicação é pequena e a separação precisa ser direta.
- Vantagens:
 - é simples;
 - reduz número de partes;
 - fácil de entender em pequenos sistemas.
- Desvantagens:
 - cliente pode ficar pesado;
 - menor flexibilidade de evolução;
 - pode acoplar interface e lógica demais;
 - escala pior que soluções mais modularizadas.
- Exemplo prático: uma aplicação local ou corporativa em que a interface acessa diretamente o banco.
- Relação com princípios de projeto: pode manter a estrutura simples, mas tende a aumentar acoplamento entre cliente e lógica.

- Frase curta para prova: "Duas camadas separa cliente e banco, mas pode concentrar lógica no cliente e limitar evolução."

Arquitetura de três camadas

- Definição: arquitetura que separa apresentação, lógica de negócio e persistência em três níveis.
- Como funciona: a interface chama a camada de negócio, que por sua vez acessa a camada de dados.
- Componentes principais: apresentação, aplicação/negócio, acesso a dados.
- Quando usar: sistemas corporativos e aplicações web em que se quer separar melhor responsabilidades.
- Vantagens:
 - separa responsabilidades;
 - facilita manutenção;
 - melhora organização;
 - reduz acoplamento entre interface e dados.
- Desvantagens:
 - aumenta estrutura;
 - pode introduzir mais camadas de comunicação;
 - pode ser exagero para sistemas pequenos.
- Exemplo prático: um sistema acadêmico com tela, serviço de matrícula e banco de dados.
- Relação com princípios de projeto: aumenta coesão por camada e favorece baixo acoplamento.
- Frase curta para prova: "Três camadas separa interface, regra de negócio e dados, deixando o sistema mais organizado."

Backend as a Service (BaaS)

- Definição: modelo em que parte do backend fica pronta como serviço externo.
- Como funciona: o aplicativo usa serviços de autenticação, banco, storage ou API fornecidos por terceiros.
- Componentes principais: cliente, serviços de backend terceirizados, integrações externas.
- Quando usar: quando se quer acelerar desenvolvimento e delegar infraestrutura de backend.
- Vantagens:
 - acelera o desenvolvimento;
 - reduz trabalho de infraestrutura;
 - facilita começar projetos menores;
 - diminui esforço inicial.
- Desvantagens:
 - dependência de fornecedor;
 - menos controle sobre o backend;
 - pode dificultar personalização;
 - pode gerar limitações de custo ou evolução.
- Exemplo prático: um app mobile que usa autenticação e armazenamento fornecidos por um serviço externo.
- Relação com princípios de projeto: reduz responsabilidade local, mas pode aumentar dependência de uma plataforma externa.
- Frase curta para prova: "BaaS terceiriza parte do backend para acelerar o desenvolvimento, mas cria dependência do provedor."

MVC

- Definição: arquitetura que separa Modelo, Visão e Controlador.
- Como funciona: a visão mostra a interface, o controlador trata eventos e o modelo concentra dados e regras.
- Componentes principais: Model, View, Controller.
- Quando usar: interfaces gráficas e aplicações web que precisam separar apresentação e lógica de interação.
- Vantagens:
 - separa responsabilidades;
 - facilita organização da interface;
 - melhora manutenção;
 - ajuda a isolar regras de negócio da apresentação.
- Desvantagens:
 - pode ficar confuso em aplicações web se a separação for mal feita;
 - pode adicionar indireção;
 - MVC clássico não foi pensado para sistemas distribuídos.
- Exemplo prático: um sistema acadêmico com página de consulta, controlador de busca e modelo com dados do aluno.
- Relação com princípios de projeto: melhora coesão, separação de responsabilidades e reduz acoplamento entre interface e regras.
- Frase curta para prova: "MVC separa interface, controle e dados, organizando melhor aplicações com interface."

Arquitetura orientada a microsserviços

- Definição: arquitetura em que módulos ou conjuntos de módulos viram processos independentes.
- Como funciona: cada microsserviço roda separadamente e pode ser implantado, escalado e evoluído de forma mais autônoma.
- Componentes principais: vários serviços pequenos, comunicação entre serviços, infraestrutura de rede e observabilidade.
- Quando usar: sistemas grandes, com times independentes e necessidade de escalabilidade por partes.
- Vantagens:
 - melhora escalabilidade;
 - dá autonomia de deployment;
 - permite tecnologias diferentes;
 - favorece falhas parciais;
 - facilita releases independentes.
- Desvantagens:
 - maior complexidade;
 - comunicação mais difícil;
 - latência de rede;
 - transações distribuídas;
 - monitoramento mais complexo;
 - implantação mais trabalhosa.
- Exemplo prático: um e-commerce com serviços separados para catálogo, pedidos, pagamento e frete.

- Relação com princípios de projeto: pode reduzir acoplamento entre partes, mas exige bom controle de responsabilidades e comunicação.
- Frase curta para prova: "Microsserviços dividem o sistema em serviços pequenos e independentes, melhorando escala e autonomia, mas aumentando complexidade."

Arquitetura orientada a mensagens

- Definição: arquitetura em que clientes e servidores se comunicam por meio de uma fila ou broker de mensagens.
- Como funciona: o emissor publica mensagens e o consumidor processa quando puder.
- Componentes principais: produtores, consumidores, fila/broker de mensagens.
- Quando usar: quando se quer comunicação assíncrona e desacoplada.
- Vantagens:
 - tolerância a falhas;
 - escalabilidade;
 - acoplamento fraco;
 - melhor desacoplamento temporal.
- Desvantagens:
 - mais complexidade de integração;
 - processamento assíncrono pode complicar depuração;
 - dependência do broker.
- Exemplo prático: um sistema de vendas publica uma mensagem e outro serviço processa faturamento ou e-mail depois.
- Relação com princípios de projeto: reduz acoplamento e separa responsabilidades entre quem publica e quem consome.
- Frase curta para prova: "Arquitetura orientada a mensagens desacopla produtor e consumidor usando um intermediário."

Publish/Subscribe

- Definição: variação da comunicação por mensagens em que eventos são publicados e assinantes recebem notificações.
- Como funciona: um sistema publica um evento e vários assinantes reagem a ele.
- Componentes principais: publicador, eventos, assinantes, broker.
- Quando usar: quando vários sistemas precisam reagir ao mesmo fato.
- Vantagens:
 - comunicação em grupo;
 - baixo acoplamento;
 - boa escalabilidade;
 - favorece reuso de consumidores.
- Desvantagens:
 - rastreamento de fluxo pode ser difícil;
 - ordem de processamento pode importar;
 - observabilidade fica mais difícil.
- Exemplo prático: venda de passagem aérea gera evento para faturamento, milhas e e-mail.
- Relação com princípios de projeto: reduz acoplamento e favorece separação de responsabilidades.

- Frase curta para prova: "Publish/Subscribe publica eventos para vários assinantes sem acoplá-los diretamente."

Pipes and Filters

- Definição: arquitetura em que filtros processam dados e os passam por pipes.
- Como funciona: cada filtro executa uma etapa e envia a saída para a próxima etapa.
- Componentes principais: filtros, pipes.
- Quando usar: quando o processamento pode ser dividido em etapas encadeadas.
- Vantagens:
 - muito flexível;
 - facilita reuso de etapas;
 - separa responsabilidades;
 - simplifica composição de processamento.
- Desvantagens:
 - pode aumentar custo de passagem de dados;
 - depuração de fluxo pode ficar mais difícil;
 - nem todo problema se encaixa bem em etapas sequenciais.
- Exemplo prático: `ls | grep csv | sort`.
- Relação com princípios de projeto: melhora separação de responsabilidades e reuso de filtros.
- Frase curta para prova: "Pipes and Filters divide o processamento em etapas independentes conectadas por fluxos."

Cliente/Servidor

- Definição: arquitetura em que um cliente solicita recursos e um servidor os fornece.
- Como funciona: o cliente envia requisições e o servidor responde com serviços ou dados.
- Componentes principais: cliente, servidor, rede.
- Quando usar: serviços web, impressão, arquivos e sistemas de rede em geral.
- Vantagens:
 - centraliza recursos;
 - simplifica manutenção do lado servidor;
 - organiza comunicação entre partes.
- Desvantagens:
 - depende da rede;
 - servidor pode virar gargalo;
 - pode existir ponto único de falha.
- Exemplo prático: navegador acessando uma API web.
- Relação com princípios de projeto: reduz acoplamento entre consumidor e provedor, mas pode concentrar responsabilidades no servidor.
- Frase curta para prova: "Cliente/Servidor separa quem pede serviço de quem fornece o serviço."

Peer-to-Peer

- Definição: arquitetura em que cada nó pode ser cliente e servidor ao mesmo tempo.
- Como funciona: os nós compartilham recursos diretamente entre si.
- Componentes principais: nós pares, comunicação direta.
- Quando usar: compartilhamento de arquivos, blockchain e sistemas distribuídos descentralizados.

- Vantagens:
 - descentralização;
 - reduz dependência de servidor central;
 - pode melhorar resiliência.
- Desvantagens:
 - coordenação mais difícil;
 - controle e segurança mais complexos;
 - comportamento menos previsível.
- Exemplo prático: BitTorrent ou blockchain.
- Relação com princípios de projeto: diminui dependência de um ponto central, mas aumenta complexidade de coordenação.
- Frase curta para prova: "Peer-to-Peer descentraliza a rede porque cada nó pode consumir e fornecer recursos."

Tabela-resumo das arquiteturas

Arquitetura	Ideia central	Quando usar	Principal vantagem	Principal desvantagem
Monolítica	Tudo roda em uma aplicação/processo	Sistemas menores ou início de projeto	Simplicidade inicial	Escala e evolução mais difíceis
Em camadas	Separação hierárquica por responsabilidades	Sistemas que precisam organizar interface, negócio e dados	Reduz complexidade	Pode ter muita indireção
Duas camadas	Cliente + servidor de banco	Sistemas simples	Fácil de entender	Cliente pode ficar pesado
Três camadas	Interface, negócio e dados separados	Sistemas corporativos/web	Melhora organização	Mais estrutura
BaaS	Backend terceirizado como serviço	Projetos que querem acelerar backend	Reduz trabalho inicial	Dependência de fornecedor
MVC	Modelo, visão e controlador	GUIs e web apps	Separa apresentação e regra	Pode gerar confusão em web
Microserviços	Serviços pequenos e independentes	Sistemas grandes e distribuídos	Escalabilidade e autonomia	Grande complexidade
Orientada a mensagens	Comunicação via fila/broker	Processamento assíncrono	Baixo acoplamento temporal	Depuração difícil
Publish/Subscribe	Eventos para assinantes	Vários consumidores do	Comunicação em grupo	Observabilidade difícil

Arquitetura	Ideia central	Quando usar	Principal vantagem	Principal desvantagem
		mesmo evento		
Pipes and Filters	Etapas encadeadas de processamento	Processamento em pipeline	Reuso de filtros	Fluxo pode ficar indireto
Cliente/Servidor	Cliente solicita, servidor responde	Sistemas de rede	Centraliza recursos	Pode virar gargalo
Peer-to-Peer	Nós são pares	Sistemas descentralizados	Menor dependência central	Coordenação complexa

Comparações importantes

Monolito x Microsserviços

- Monolito: uma aplicação/processo único.
- Microsserviços: vários serviços/processos independentes.
- Diferença principal: o monolito concentra tudo; microsserviços distribuem partes do sistema.
- Quando escolher cada um:
 - Monolito: quando o sistema é pequeno ou quando a prioridade é simplicidade.
 - Microsserviços: quando há necessidade real de escala, autonomia e times independentes.

Duas camadas x Três camadas

- Duas camadas: cliente + banco de dados.
- Três camadas: apresentação + negócio + dados.
- Diferença principal: três camadas separa melhor as responsabilidades.
- Quando escolher cada uma:
 - Duas camadas: sistemas simples.
 - Três camadas: sistemas que precisam de melhor organização e manutenção.

MVC x Camadas

- MVC: separa modelo, visão e controlador.
- Camadas: separa apresentação, negócio e dados.
- Diferença principal: MVC organiza a interface; camadas organizam o sistema como um todo.
- Relação entre elas: MVC pode existir dentro de uma arquitetura em camadas.

BaaS x Backend próprio

- BaaS: backend terceirizado como serviço.
- Backend próprio: a equipe implementa e controla o backend.
- Diferença principal: BaaS terceiriza parte da infraestrutura e do desenvolvimento.
- Vantagens e limitações:
 - BaaS: acelera o início, mas cria dependência.
 - Backend próprio: dá mais controle, mas exige mais trabalho.

Orientada a mensagens x Publish/Subscribe

- Orientada a mensagens: comunicação por fila/broker entre produtores e consumidores.
- Publish/Subscribe: eventos publicados para múltiplos assinantes.
- Diferença principal: pub/sub é uma forma mais orientada a eventos da comunicação por mensagens.
- Quando escolher cada um:
 - Orientada a mensagens: quando a prioridade é desacoplar produtor e consumidor.
 - Publish/Subscribe: quando vários sistemas devem reagir ao mesmo evento.

Como identificar a arquitetura em questões práticas

1. Identifique se o sistema é local, web, mobile ou distribuído.
2. Verifique se há cliente, servidor e banco de dados.
3. Verifique se existe separação em camadas.
4. Verifique se há serviços independentes.
5. Justifique com base em acoplamento, responsabilidades e comunicação.

Exemplos curtos:

- Microsoft Excel desktop: tende a ser monolito, com foco em aplicação local e interface integrada.
- Aplicativo bancário mobile: normalmente cliente/servidor, com app no cliente e backend remoto.
- Sistema web de e-commerce: costuma usar MVC e camadas no backend.
- API com vários serviços: tende a microsserviços ou arquitetura distribuída por serviços.
- Sistema simples com interface acessando banco diretamente: lembra duas camadas.